MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 20 — Itérateurs et Générateurs

Paresse et élégance — produire les données à la demande

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

18 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 20 — Itérateurs et Générateurs **2**
0.1 Le protocole itérateur 2
0.2 Créer son propre itérateur 3
0.3 Les générateurs — yield 4
0.4 Les expressions génératrices 5
0.5 Générateurs avec plusieurs yield 6
0.6 yield from — Déléguer à un sous-générateur 7
0.7 Le module itertools 8
0.8 Générateurs avancés — send() et throw() 9
0.9 Résumé du Chapitre 20 10

Chapitre 20 — Itérateurs et Générateurs

À retenir

Un **itérateur** est un objet qui produit des valeurs une par une, à la demande. Un **générateur** est une façon élégante de créer des itérateurs avec `yield`. Ces concepts permettent de traiter des **séquences infinies** ou des **fichiers de plusieurs gigaoctets** sans jamais tout charger en mémoire. C'est l'une des fonctionnalités les plus puissantes et les plus pythoniques du langage.

Le protocole itérateur

En Python, tout objet que l'on peut parcourir avec `for` implémente le **protocole itérateur**.

Comprendre le protocole itérateur

```python
# Tout for cache en realite un iterateur
fruits = ["pomme", "banane", "cerise"]

# Ce que Python fait VRAIMENT derriere un for :
iterateur = iter(fruits)          # appelle fruits.__iter__()
print(next(iterateur))            # pomme
print(next(iterateur))            # banane
print(next(iterateur))            # cerise
# print(next(iterateur))          # StopIteration !

# Le for gere StopIteration automatiquement
for fruit in fruits:
    print(fruit)

# iter() fonctionne sur tout iterable
print(next(iter("Python")))       # P
print(next(iter(range(10))))      # 0
print(next(iter({1, 2, 3})))      # valeur quelconque

# Verifier si un objet est iterable
from collections.abc import Iterable, Iterator

print(isinstance([1,2,3],  Iterable))   # True
print(isinstance("Python", Iterable))   # True
print(isinstance(42,       Iterable))   # False

# Difference iterable vs iterateur
liste = [1, 2, 3]
it    = iter(liste)
```

```
print(isinstance(liste, Iterator))   # False      iterable
print(isinstance(it,    Iterator))   # True       iterateur
```

Créer son propre itérateur

Un itérateur personnalisé implémente `__iter__` et `__next__`.

Classe itérateur — compte à rebours

```
class CompteARebours:
    """Iterateur qui compte de n a 0."""

    def __init__(self, debut):
        self.debut   = debut
        self.courant = debut

    def __iter__(self):
        # Retourne l'iterateur lui-m me
        return self

    def __next__(self):
        if self.courant < 0:
            raise StopIteration   # signal d'arret
        valeur        = self.courant
        self.courant -= 1
        return valeur


# Utilisation
decompte = CompteARebours(5)

for n in decompte:
    print(n, end=" ")
print("D collage !")
#     5 4 3 2 1 0 Decollage !

# Utilisation avec next()
decompte2 = CompteARebours(3)
print(next(decompte2))   # 3
print(next(decompte2))   # 2
print(next(decompte2))   # 1
print(next(decompte2))   # 0
# print(next(decompte2)) # StopIteration

# Convertir en liste
print(list(CompteARebours(5)))   # [5, 4, 3, 2, 1, 0]

# Utiliser dans une comprehension
carres = [x**2 for x in CompteARebours(4)]
```

```python
print(carres)    # [16, 9, 4, 1, 0]
```

Les générateurs — yield

Un **générateur** est une fonction qui utilise `yield` au lieu de `return`. Il produit les valeurs une par une, en **suspendant** son exécution entre chaque `yield`.

yield — créer un générateur

```python
# Fonction classique      retourne TOUT d'un coup
def carre_liste(n):
    return [i**2 for i in range(n)]

# Generateur      produit UNE valeur a la fois
def carre_generateur(n):
    for i in range(n):
        yield i**2    # suspend ici, reprend au next() suivant

# Comparaison memoire
import sys
liste = carre_liste(1000)
gen   = carre_generateur(1000)

print(f"Liste      : {sys.getsizeof(liste):,} octets")
print(f"G n rateur: {sys.getsizeof(gen):,} octets")
# Liste      : 8,856 octets
# G n rateur:    112 octets       80x moins !

# Le generateur se comporte comme un iterateur
gen2 = carre_generateur(5)
print(next(gen2))    # 0
print(next(gen2))    # 1
print(next(gen2))    # 4

# Parcourir avec for
for valeur in carre_generateur(5):
    print(valeur, end=" ")
#      0 1 4 9 16

# Exemples pratiques
def fibonacci():
    """Generateur infini de la suite de Fibonacci."""
    a, b = 0, 1
    while True:    # infini !
        yield a
        a, b = b, a + b

# Prendre les 10 premiers
fib = fibonacci()
```

```python
for _ in range(10):
    print(next(fib), end=" ")
#     0 1 1 2 3 5 8 13 21 34
```

À retenir

`yield` **suspend** l'exécution de la fonction et retourne la valeur. Au prochain `next()`, l'exécution **reprend** exactement là où elle s'était arrêtée — avec toutes les variables locales intactes. C'est fondamentalement différent de `return` qui termine la fonction définitivement.

Les expressions génératrices

Comme les list comprehensions, les **expressions génératrices** créent des générateurs en une ligne.

Expressions génératrices — syntaxe

```python
# List comprehension      cree une liste en memoire
carres_liste = [x**2 for x in range(1000000)]

# Expression generatrice      cree un generateur paresseux
carres_gen = (x**2 for x in range(1000000))

import sys
print(sys.getsizeof(carres_liste))   # ~8 Mo
print(sys.getsizeof(carres_gen))     # 112 octets !

# Syntaxe : (expression for x in iterable if condition)
pairs_gen = (x for x in range(100) if x % 2 == 0)

# Utilisation directe dans sum(), max(), min()
total = sum(x**2 for x in range(101))
print(f"Somme des carres : {total}")   # 338350

maximum = max(len(mot) for mot in ["Python", "est", "incroyable
    "])
print(f"Mot le plus long : {maximum} lettres")   # 9

# Chainer des expressions generatrices
nombres = range(1, 101)
resultat = sum(
    x**2
    for x in nombres
    if x % 2 == 0   # pairs seulement
)
print(f"Somme des carres pairs : {resultat}")   # 171700
```

```python
# Generateur de lignes d'un fichier (tres efficace !)
def lire_lignes_non_vides(nom_fichier):
    with open(nom_fichier, "r", encoding="utf-8") as f:
        return (ligne.strip() for ligne in f if ligne.strip())
```

Générateurs avec plusieurs yield

Générateurs complexes — yield multiples

```python
def lire_par_blocs(fichier, taille_bloc=1024):
    """Lit un fichier par blocs     efficace pour les gros
       fichiers."""
    with open(fichier, "rb") as f:
        while True:
            bloc = f.read(taille_bloc)
            if not bloc:
                break
            yield bloc

def progresser(iterable, total=None):
    """Affiche la progression d'une iteration."""
    for i, element in enumerate(iterable, 1):
        if total:
            pct = i / total * 100
            print(f"\r  Progression : {pct:.1f}%", end="")
        yield element
    print()   # nouvelle ligne a la fin

def generer_rapport(etudiants):
    """Genere les lignes d'un rapport une par une."""
    yield "=" * 50
    yield f"RAPPORT     {len(etudiants)}   tudiants  "
    yield "=" * 50

    admis    = 0
    total    = 0
    for nom, note in etudiants:
        mention = "TB" if note >= 16 else \
                  "B"  if note >= 14 else \
                  "AB" if note >= 12 else \
                  "P"  if note >= 10 else "F"
        statut = "   " if note >= 10 else "   "
        yield f"  {statut} {nom:<12} {note:3}/20  {mention}"
        total += note
        if note >= 10:
            admis += 1

    yield "-" * 50
```

```python
    yield f"  Moyenne : {total/len(etudiants):.1f}/20"
    yield f"  Admis   : {admis}/{len(etudiants)}"
    yield "=" * 50


etudiants = [
    ("Alice",   15), ("Bob",     18),
    ("Charlie", 12), ("Diana",    9),
    ("Eve",     17), ("Frank",    7),
]

for ligne in generer_rapport(etudiants):
    print(ligne)
```

yield from — Déléguer à un sous-générateur

yield from — composition de générateurs

```python
# yield from delegue a un autre iterable ou generateur

def chiffres():
    yield from range(10)          # delegue a range

def alphabet():
    yield from "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def combiner(*iterables):
    """Combine plusieurs iterables en un seul generateur."""
    for iterable in iterables:
        yield from iterable

# Utilisation
print(list(chiffres()))
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

for c in alphabet():
    print(c, end="")
print()

print(list(combiner([1,2,3], "abc", (True, False))))
# [1, 2, 3, 'a', 'b', 'c', True, False]

# Aplatir une structure imbriquee
def aplatir(structure):
    """Aplatit recursivement une liste imbriquee."""
    for element in structure:
        if isinstance(element, list):
            yield from aplatir(element)    # recursion !
```

```
        else:
            yield element

imbrique = [1, [2, 3, [4, 5]], 6, [7, [8, 9]]]
print(list(aplatir(imbrique)))
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Le module itertools

`itertools` est une bibliothèque de générateurs haute performance pour les opérations sur les itérables.

itertools — les incontournables

```
import itertools

# count()      compteur infini
compteur = itertools.count(start=1, step=2)
print([next(compteur) for _ in range(5)])
# [1, 3, 5, 7, 9]

# cycle()      repeter indefiniment
couleurs = itertools.cycle(["rouge", "vert", "bleu"])
print([next(couleurs) for _ in range(7)])
# ['rouge', 'vert', 'bleu', 'rouge', 'vert', 'bleu', 'rouge']

# repeat()      repeter n fois
print(list(itertools.repeat("Python", 3)))

# chain()      enchainer des iterables
print(list(itertools.chain([1,2], [3,4], [5,6])))
# [1, 2, 3, 4, 5, 6]

# islice()      trancher un iterateur
fib = (a for a, _ in itertools.accumulate(
    zip(itertools.repeat(0), itertools.repeat(1)),
    lambda ab, _: (ab[1], ab[0]+ab[1])
))
# Plus simple avec notre generateur :
def fib_gen():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

premiers_20 = list(itertools.islice(fib_gen(), 20))
print(premiers_20)

# takewhile()      tant que la condition est vraie
```

```python
petits = list(itertools.takewhile(lambda x: x < 50, fib_gen()))
print(petits)   # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

# dropwhile()     sauter tant que condition vraie
grands = list(itertools.islice(
    itertools.dropwhile(lambda x: x < 50, fib_gen()), 5
))
print(grands)   # [55, 89, 144, 233, 377]

# combinations() et permutations()
print(list(itertools.combinations("ABC", 2)))
# [('A','B'), ('A','C'), ('B','C')]

print(list(itertools.permutations("AB", 2)))
# [('A','B'), ('B','A')]

# product()     produit cartesien
print(list(itertools.product([1,2], ["a","b"])))
# [(1,'a'), (1,'b'), (2,'a'), (2,'b')]
```

Générateurs avancés — send() et throw()

Générateurs bidirectionnels avec send()

```python
def accumulateur():
    """Generateur qui recoit des valeurs et accumule."""
    total = 0
    while True:
        valeur = yield total   # yield retourne total ET re oit valeur
        if valeur is None:
            break
        total += valeur
        print(f"  Re u {valeur}, total = {total}")

# Utilisation
acc = accumulateur()
next(acc)         # initialiser     avancer jusqu'au premier yield

acc.send(10)      # envoyer 10
acc.send(25)      # envoyer 25
acc.send(5)       # envoyer 5
resultat = acc.send(0)
print(f"Total final : {resultat}")

# Exemple pratique : pipeline de traitement
def pipeline_donnees():
```

```
    """Generateur qui traite des donnees en flux."""
    resultats = []
    while True:
        donnee = yield
        if donnee is None:
            yield resultats
            return
        # Traitement
        if isinstance(donnee, (int, float)) and donnee > 0:
            resultats.append(donnee * 2)
```

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `yield` suspend — il ne termine pas. La distinction entre `return` (termine la fonction) et `yield` (suspend et reprend) m'a pris du temps à intégrer. Un générateur est une **coroutine légère** — son état d'exécution est préservé entre les appels. Les variables locales, la position dans le code, tout est gardé. C'est comme mettre une fonction en pause.

2. La différence de mémoire est spectaculaire. Générer un million de carrés avec une liste prend plusieurs mégaoctets. Avec un générateur : 112 octets. Cette différence de plusieurs ordres de grandeur change radicalement ce qu'un programme peut faire. Traiter un fichier de logs de 50 Go ligne par ligne avec un générateur — impossible avec une liste.

3. `itertools` est une mine d'or sous-estimée. `chain()`, `islice()`, `takewhile()`, `combinations()` — toutes ces fonctions retournent des générateurs paresseux, combinables à l'infini. Découvrir `itertools`, c'est réaliser que Python pense la manipulation de données comme une **composition de flux**, pas comme une accumulation en mémoire.

Résumé du Chapitre 20

Ce que j'ai appris dans ce chapitre :

- **Protocole itérateur** : `__iter__` + `__next__` + `StopIteration`.
- `iter()` et `next()` — les fonctions built-in derrière tout `for`.
- `yield` — suspendre une fonction et produire une valeur.
- **Générateurs** : mémoire constante quelle que soit la taille des données.
- **Expressions génératrices** : `(expr for x in iter if cond)`.
- `yield from` — déléguer à un sous-générateur ou itérable.
- **itertools** : `count`, `cycle`, `chain`, `islice`, `takewhile`...

`send()` — envoyer des valeurs dans un générateur bidirectionnel.

"Un générateur ne calcule pas tout d'avance. Il calcule ce dont on a besoin, quand on en a besoin. C'est la philosophie de la paresse intelligente."